

ola

April 29, 2026

```
[ ]: import numpy as np
import pandas as pd
```

```
[ ]: from google.colab import files
uploaded = files.upload()
```

<IPython.core.display.HTML object>

Saving ola_driver.csv to ola_driver (2).csv

```
[ ]: df = pd.read_csv("ola_driver.csv")
df.head()
```

/usr/local/lib/python3.12/dist-

packages/google/colab/_dataframe_summarizer.py:88: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
cast_date_col = pd.to_datetime(column, errors="coerce")
```

```
[ ]: Unnamed: 0    MMM-YY  Driver_ID  Age  Gender  City  Education_Level  \
0          0    01/01/19          1  28.0    0.0  C23                2
1          1    02/01/19          1  28.0    0.0  C23                2
2          2    03/01/19          1  28.0    0.0  C23                2
3          3   11/01/20          2  31.0    0.0   C7                 2
4          4   12/01/20          2  31.0    0.0   C7                 2
```

```
Income Dateofjoining LastWorkingDate  Joining Designation  Grade  \
0   57387    24/12/18          NaN          1          1
1   57387    24/12/18          NaN          1          1
2   57387    24/12/18    03/11/19          1          1
3   67016    11/06/20          NaN          2          2
4   67016    11/06/20          NaN          2          2
```

```
Total Business Value  Quarterly Rating
0          2381060          2
1         -665480          2
2           0          2
3           0          1
4           0          1
```

```
[ ]: df.shape
df.columns
```

```
[ ]: Index(['Unnamed: 0', 'MMM-YY', 'Driver_ID', 'Age', 'Gender', 'City',
          'Education_Level', 'Income', 'Dateofjoining', 'LastWorkingDate',
          'Joining Designation', 'Grade', 'Total Business Value',
          'Quarterly Rating'],
          dtype='object')
```

The dataset contains X rows and Y columns

```
[ ]: df.describe()
```

```
[ ]:
      Unnamed: 0      Driver_ID      Age      Gender \
count  19104.000000  19104.000000  19043.000000  19052.000000
mean    9551.500000   1415.591133    34.668435    0.418749
std     5514.994107    810.705321     6.257912    0.493367
min         0.000000     1.000000    21.000000    0.000000
25%     4775.750000    710.000000    30.000000    0.000000
50%     9551.500000   1417.000000    34.000000    0.000000
75%    14327.250000   2137.000000    39.000000    1.000000
max    19103.000000   2788.000000    58.000000    1.000000

      Education_Level      Income      Joining Designation      Grade \
count    19104.000000   19104.000000   19104.000000  19104.000000
mean         1.021671   65652.025126         1.690536    2.252670
std         0.800167   30914.515344         0.836984    1.026512
min         0.000000   10747.000000         1.000000    1.000000
25%         0.000000   42383.000000         1.000000    1.000000
50%         1.000000   60087.000000         1.000000    2.000000
75%         2.000000   83969.000000         2.000000    3.000000
max         2.000000  188418.000000         5.000000    5.000000

      Total Business Value      Quarterly Rating
count    1.910400e+04    19104.000000
mean     5.716621e+05      2.008899
std     1.128312e+06      1.009832
min     -6.000000e+06      1.000000
25%      0.000000e+00      1.000000
50%      2.500000e+05      2.000000
75%      6.997000e+05      3.000000
max      3.374772e+07      4.000000
```

the min age of driver is 21 the avg income of driver is 10747

```
[ ]: df.isnull().sum()
```

```
[ ]: Unnamed: 0      0
      MMM-YY      0
      Driver_ID    0
      Age         61
      Gender      52
      City        0
      Education_Level 0
      Income      0
      Dateofjoining 0
      LastWorkingDate 17488
      Joining Designation 0
      Grade       0
      Total Business Value 0
      Quarterly Rating 0
      dtype: int64
```

```
[ ]: df['Driver_ID'].nunique()
```

```
[ ]: 2381
```

Total unique drivers in data is 2318

```
[ ]: df['Age'].fillna(df['Age'].mean(), inplace=True)
      df
```

/tmp/ipykernel_687/1943981980.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
/usr/local/lib/python3.12/dist-
packages/google/colab/_dataframe_summarizer.py:88: UserWarning: Could not infer
format, so each element will be parsed individually, falling back to `dateutil`.
To ensure parsing is consistent and as-expected, please specify a format.
cast_date_col = pd.to_datetime(column, errors="coerce")
```

```
[ ]: Unnamed: 0      MMM-YY  Driver_ID  Age  Gender  City  Education_Level  \
0      0      01/01/19      1  28.0      0.0  C23      2
1      1      02/01/19      1  28.0      0.0  C23      2
2      2      03/01/19      1  28.0      0.0  C23      2
```

3	3	11/01/20	2	31.0	0.0	C7	2
4	4	12/01/20	2	31.0	0.0	C7	2
...	...	...	...	...	...	...	...
19099	19099	08/01/20	2788	30.0	0.0	C27	2
19100	19100	09/01/20	2788	30.0	0.0	C27	2
19101	19101	10/01/20	2788	30.0	0.0	C27	2
19102	19102	11/01/20	2788	30.0	0.0	C27	2
19103	19103	12/01/20	2788	30.0	0.0	C27	2

	Income	Dateofjoining	LastWorkingDate	Joining	Designation	Grade	\
0	57387	24/12/18	NaN			1	1
1	57387	24/12/18	NaN			1	1
2	57387	24/12/18	03/11/19			1	1
3	67016	11/06/20	NaN			2	2
4	67016	11/06/20	NaN			2	2
...	...	...	...	...	...	...	...
19099	70254	06/08/20	NaN			2	2
19100	70254	06/08/20	NaN			2	2
19101	70254	06/08/20	NaN			2	2
19102	70254	06/08/20	NaN			2	2
19103	70254	06/08/20	NaN			2	2

	Total Business Value	Quarterly Rating
0	2381060	2
1	-665480	2
2	0	2
3	0	1
4	0	1
...	...	...
19099	740280	3
19100	448370	3
19101	0	2
19102	200420	2
19103	411480	2

[19104 rows x 14 columns]

```
[ ]: df['Gender'].fillna(df['Gender'].mode()[0], inplace=True)
df
```

/tmp/ipykernel_687/1825244161.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using

'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.

```
df['Gender'].fillna(df['Gender'].mode()[0], inplace=True)
/usr/local/lib/python3.12/dist-
packages/google/colab/_dataframe_summarizer.py:88: UserWarning: Could not infer
format, so each element will be parsed individually, falling back to `dateutil`.
To ensure parsing is consistent and as-expected, please specify a format.
cast_date_col = pd.to_datetime(column, errors="coerce")
```

```
[ ]:      Unnamed: 0      MMM-YY  Driver_ID  Age  Gender City  Education_Level  \
0          0  01/01/19          1  28.0    0.0  C23          2
1          1  02/01/19          1  28.0    0.0  C23          2
2          2  03/01/19          1  28.0    0.0  C23          2
3          3  11/01/20          2  31.0    0.0   C7          2
4          4  12/01/20          2  31.0    0.0   C7          2
...
19099      19099  08/01/20      2788  30.0    0.0  C27          2
19100      19100  09/01/20      2788  30.0    0.0  C27          2
19101      19101  10/01/20      2788  30.0    0.0  C27          2
19102      19102  11/01/20      2788  30.0    0.0  C27          2
19103      19103  12/01/20      2788  30.0    0.0  C27          2
```

```
      Income Dateofjoining LastWorkingDate  Joining Designation  Grade  \
0      57387      24/12/18          NaN          1          1
1      57387      24/12/18          NaN          1          1
2      57387      24/12/18      03/11/19          1          1
3      67016      11/06/20          NaN          2          2
4      67016      11/06/20          NaN          2          2
...
19099      70254      06/08/20          NaN          2          2
19100      70254      06/08/20          NaN          2          2
19101      70254      06/08/20          NaN          2          2
19102      70254      06/08/20          NaN          2          2
19103      70254      06/08/20          NaN          2          2
```

```
      Total Business Value  Quarterly Rating
0          2381060          2
1         -665480          2
2              0          2
3              0          1
4              0          1
...
19099          740280          3
19100          448370          3
19101              0          2
```

19102	200420	2
19103	411480	2

[19104 rows x 14 columns]

```
[ ]: df.isnull().sum()
```

```
[ ]: Unnamed: 0      0
      MMM-YY      0
      Driver_ID    0
      Age          0
      Gender       0
      City         0
      Education_Level 0
      Income       0
      Dateofjoining 0
      LastWorkingDate 17488
      Joining Designation 0
      Grade        0
      Total Business Value 0
      Quarterly Rating 0
      dtype: int64
```

```
[ ]: df['HasLeft'] = df['LastWorkingDate'].notna().astype(int)
      df
```

```
/usr/local/lib/python3.12/dist-
packages/google/colab/_dataframe_summarizer.py:88: UserWarning: Could not infer
format, so each element will be parsed individually, falling back to `dateutil`.
To ensure parsing is consistent and as-expected, please specify a format.
      cast_date_col = pd.to_datetime(column, errors="coerce")
```

```
[ ]:      Unnamed: 0      MMM-YY  Driver_ID   Age  Gender City  Education_Level  \
0          0      0 01/01/19          1  28.0    0.0  C23          2
1          1      1 02/01/19          1  28.0    0.0  C23          2
2          2      2 03/01/19          1  28.0    0.0  C23          2
3          3      3 11/01/20          2  31.0    0.0   C7          2
4          4      4 12/01/20          2  31.0    0.0   C7          2
...      ...      ...      ...      ...      ...      ...
19099      19099 08/01/20          2788  30.0    0.0  C27          2
19100      19100 09/01/20          2788  30.0    0.0  C27          2
19101      19101 10/01/20          2788  30.0    0.0  C27          2
19102      19102 11/01/20          2788  30.0    0.0  C27          2
19103      19103 12/01/20          2788  30.0    0.0  C27          2

      Income Dateofjoining LastWorkingDate  Joining Designation  Grade  \
0      57387      24/12/18          NaN          1          1
1      57387      24/12/18          NaN          1          1
```

2	57387	24/12/18	03/11/19		1	1
3	67016	11/06/20	NaN		2	2
4	67016	11/06/20	NaN		2	2
...	...	...	...	...	...	...
19099	70254	06/08/20	NaN		2	2
19100	70254	06/08/20	NaN		2	2
19101	70254	06/08/20	NaN		2	2
19102	70254	06/08/20	NaN		2	2
19103	70254	06/08/20	NaN		2	2

	Total Business Value	Quarterly Rating	HasLeft
0	2381060	2	0
1	-665480	2	0
2	0	2	1
3	0	1	0
4	0	1	0
...	...	...	...
19099	740280	3	0
19100	448370	3	0
19101	0	2	0
19102	200420	2	0
19103	411480	2	0

[19104 rows x 15 columns]

The HasLeft column now indicates whether a driver has left the company (1) or not (0).

```
[ ]: df['MMM-YY'] = pd.to_datetime(df['MMM-YY'])
df['Dateofjoining'] = pd.to_datetime(df['Dateofjoining'])
df['LastWorkingDate'] = pd.to_datetime(df['LastWorkingDate'])
df
```

/tmp/ipykernel_687/4183973159.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
df['MMM-YY'] = pd.to_datetime(df['MMM-YY'])
```

/tmp/ipykernel_687/4183973159.py:2: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
df['Dateofjoining'] = pd.to_datetime(df['Dateofjoining'])
```

/tmp/ipykernel_687/4183973159.py:3: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
df['LastWorkingDate'] = pd.to_datetime(df['LastWorkingDate'])
```

```
[ ]: Unnamed: 0    MMM-YY  Driver_ID  Age  Gender  City  Education_Level  \
0          0  2019-01-01          1  28.0    0.0  C23              2
1          1  2019-02-01          1  28.0    0.0  C23              2
```

2	2	2019-03-01	1	28.0	0.0	C23	2
3	3	2020-11-01	2	31.0	0.0	C7	2
4	4	2020-12-01	2	31.0	0.0	C7	2
...	...	...	...	...	...	...	...
19099	19099	2020-08-01	2788	30.0	0.0	C27	2
19100	19100	2020-09-01	2788	30.0	0.0	C27	2
19101	19101	2020-10-01	2788	30.0	0.0	C27	2
19102	19102	2020-11-01	2788	30.0	0.0	C27	2
19103	19103	2020-12-01	2788	30.0	0.0	C27	2

	Income	Dateofjoining	LastWorkingDate	Joining	Designation	Grade	\
0	57387	2018-12-24	NaT			1	1
1	57387	2018-12-24	NaT			1	1
2	57387	2018-12-24	2019-03-11			1	1
3	67016	2020-11-06	NaT			2	2
4	67016	2020-11-06	NaT			2	2
...	...	...	...	...	...	...	...
19099	70254	2020-06-08	NaT			2	2
19100	70254	2020-06-08	NaT			2	2
19101	70254	2020-06-08	NaT			2	2
19102	70254	2020-06-08	NaT			2	2
19103	70254	2020-06-08	NaT			2	2

	Total Business Value	Quarterly Rating	HasLeft
0	2381060	2	0
1	-665480	2	0
2	0	2	1
3	0	1	0
4	0	1	0
...	...	...	...
19099	740280	3	0
19100	448370	3	0
19101	0	2	0
19102	200420	2	0
19103	411480	2	0

[19104 rows x 15 columns]

Error: Runtime no longer has a reference to this dataframe, please re-run this cell and try again.

Error: Runtime no longer has a reference to this dataframe, please re-run this cell and try again.

Error: Runtime no longer has a reference to this dataframe, please re-run this cell and try again.

Error: Runtime no longer has a reference to this dataframe, please re-run this cell and try again.

CONVERT DATES

```
[ ]: df.groupby(df['Dateofjoining'].dt.to_period('M'))['Driver_ID'].count()
```

```
[ ]: Dateofjoining
2013-04      31
2013-05      24
2013-06      59
2013-07      63
2013-08      33
...
2020-08     325
2020-09     314
2020-10     139
2020-11      93
2020-12      59
Freq: M, Name: Driver_ID, Length: 85, dtype: int64
```

Drivers joined per month

```
[ ]: df.groupby(df['LastWorkingDate'].dt.to_period('M'))['Driver_ID'].count()
```

```
[ ]: LastWorkingDate
2018-12       5
2019-01      82
2019-02      85
2019-03      75
2019-04      49
2019-05      98
2019-06      61
2019-07      48
2019-08      53
2019-09      79
2019-10      70
2019-11      69
2019-12      56
2020-01      70
2020-02      70
2020-03      58
2020-04      42
2020-05      63
2020-06      77
2020-07     141
2020-08       4
2020-09      66
2020-10      62
2020-11      73
2020-12      60
Freq: M, Name: Driver_ID, dtype: int64
```

Drivers left per month

```
[ ]: df['tenure'] = (df['LastWorkingDate'].fillna(df['MMM-YY'])-df['Dateofjoining']).  
    ↪dt.days  
df['tenure'].mean() / 365
```

```
[ ]: np.float64(1.6222437816938575)
```

The average tenure of drivers is approximately 592 days 1.6 years, indicating that drivers typically remain with the company for more than one year.

```
[ ]: df['Attrition_flag'] = df['LastWorkingDate'].notnull().astype(int)
```

Target Variable

```
[ ]: df['Attrition_flag'].value_counts()
```

```
[ ]: Attrition_flag  
0    17488  
1     1616  
Name: count, dtype: int64
```

```
[ ]: df['tenure_days'] = (df['LastWorkingDate'].fillna(df['MMM-YY']) -  
    ↪df['Dateofjoining']).dt.days  
df['tenure_days']
```

```
[ ]: 0         8  
     1        39  
     2        77  
     3        -5  
     4        25  
     ...  
19099        54  
19100        85  
19101       115  
19102       146  
19103       176  
Name: tenure_days, Length: 19104, dtype: int64
```

Tenure Feature

```
[ ]: df['Joining_year'] = df['Dateofjoining'].dt.year  
df['Joining_year']
```

```
[ ]: 0        2018  
     1        2018  
     2        2018  
     3        2020  
     4        2020
```

```

...
19099    2020
19100    2020
19101    2020
19102    2020
19103    2020
Name: Joining_year, Length: 19104, dtype: int32

```

joining year

```

[ ]: df = df.sort_values(by=['Driver_ID', 'MMM-YY'])

[ ]: df['Rating_increased'] = df.groupby('Driver_ID')['Quarterly Rating'].diff().
      ↪gt(0).astype(int)
      df['Rating_increased']

```

```

[ ]: 0      0
      1      0
      2      0
      3      0
      4      0
      ..
19099    0
19100    0
19101    0
19102    0
19103    0
Name: Rating_increased, Length: 19104, dtype: int64

```

A feature was made to keep track of how drivers' performance is getting better by looking at ratings from the last three months. A value of 1 means that ratings improving, while a value of 0 means ratings declining.

```

[ ]: df['Income_increased'] = df.groupby('Driver_ID')['Income'].diff().gt(0).
      ↪astype(int)
      df['Income_increased']

```

```

[ ]: 0      0
      1      0
      2      0
      3      0
      4      0
      ..
19099    0
19100    0
19101    0
19102    0
19103    0

```

Name: Income_increased, Length: 19104, dtype: int64

```
[ ]: df_final = df.groupby('Driver_ID').agg({
    'Age': 'last',
    'Income': 'last',
    'tenure_days': 'max',
    'Rating_increased': 'sum',
    'Gender': 'last',
    'City': 'last',
    'Education_Level': 'last',
    'Joining Designation': 'last',
    'Grade': 'last',
    'Total Business Value': 'sum',
    'Quarterly Rating': 'mean',
    'Attrition_flag': 'max',
    'Income_increased': 'sum'
}).reset_index()
```

```
[ ]: df_final.shape
df_final.head()
```

```
[ ]: 
```

	Driver_ID	Age	Income	tenure_days	Rating_increased	Gender	City	\
0	1	28.0	57387	77	0	0.0	C23	
1	2	31.0	67016	25	0	0.0	C7	
2	4	43.0	65603	142	0	0.0	C13	
3	5	29.0	46368	57	0	0.0	C9	
4	6	31.0	78728	123	1	1.0	C11	

	Education_Level	Joining Designation	Grade	Total Business Value	\
0	2	1	1	1715580	
1	2	2	2	0	
2	2	2	2	350000	
3	0	1	1	120360	
4	1	3	3	1265000	

	Quarterly Rating	Attrition_flag	Income_increased
0	2.0	1	0
1	1.0	0	0
2	1.0	1	0
3	1.0	1	0
4	1.6	0	0

The dataset had more than one record per driver for each month, so it was combined into a driver-level dataset so that it could be analysed and modelled in a useful way.

```
[ ]: df_final.isnull().sum()
```

```

[ ]: Driver_ID      0
    Age             0
    Income           0
    tenure_days      0
    Rating_increased 0
    Gender           0
    Grade            0
    Total Business Value 0
    Quarterly Rating 0
    Attrition_flag   0
    Income_increased 0
    City_C10         0
    City_C11         0
    City_C12         0
    City_C13         0
    City_C14         0
    City_C15         0
    City_C16         0
    City_C17         0
    City_C18         0
    City_C19         0
    City_C2          0
    City_C20         0
    City_C21         0
    City_C22         0
    City_C23         0
    City_C24         0
    City_C25         0
    City_C26         0
    City_C27         0
    City_C28         0
    City_C29         0
    City_C3          0
    City_C4          0
    City_C5          0
    City_C6          0
    City_C7          0
    City_C8          0
    City_C9          0
    Education_Level_1 0
    Education_Level_2 0
    Joining Designation_2 0
    Joining Designation_3 0
    Joining Designation_4 0
    Joining Designation_5 0
dtype: int64

```

```
[ ]: df_final.columns
```

```
[ ]: Index(['Driver_ID', 'Age', 'Income', 'tenure_days', 'Rating_increased',
          'Gender', 'Grade', 'Total Business Value', 'Quarterly Rating',
          'Attrition_flag', 'Income_increased', 'City_C10', 'City_C11',
          'City_C12', 'City_C13', 'City_C14', 'City_C15', 'City_C16', 'City_C17',
          'City_C18', 'City_C19', 'City_C2', 'City_C20', 'City_C21', 'City_C22',
          'City_C23', 'City_C24', 'City_C25', 'City_C26', 'City_C27', 'City_C28',
          'City_C29', 'City_C3', 'City_C4', 'City_C5', 'City_C6', 'City_C7',
          'City_C8', 'City_C9', 'Education_Level_1', 'Education_Level_2',
          'Joining Designation_2', 'Joining Designation_3',
          'Joining Designation_4', 'Joining Designation_5'],
          dtype='object')
```

```
[ ]: from sklearn.preprocessing import StandardScaler
      scaler = StandardScaler()
      num_cols = ['Age', 'Income', 'Total Business Value', 'tenure_days']
      df_final[num_cols] = scaler.fit_transform(df_final[num_cols])
      df_final.head()
```

```
[ ]: Driver_ID      Age      Income  tenure_days  Rating_increased  Gender  \
0          1 -0.952941 -0.068616    -0.615893             0      0.0
1          2 -0.450207  0.270700    -0.708045             0      0.0
2          4  1.560730  0.220907    -0.500703             0      0.0
3          5 -0.785363 -0.456914    -0.651336             0      0.0
4          6 -0.450207  0.683418    -0.534374             1      1.0

      Grade  Total Business Value  Quarterly Rating  Attrition_flag  ...  \
0          1          -0.314641             2.0             1      ...
1          2          -0.502646             1.0             0      ...
2          2          -0.464290             1.0             1      ...
3          1          -0.489456             1.0             1      ...
4          3          -0.364019             1.6             0      ...

      City_C6  City_C7  City_C8  City_C9  Education_Level_1  Education_Level_2  \
0      False      False      False      False             False             True
1      False       True      False      False             False             True
2      False      False      False      False             False             True
3      False      False      False       True             False             False
4      False      False      False      False             True             False

      Joining Designation_2  Joining Designation_3  Joining Designation_4  \
0              False              False              False
1              True              False              False
2              True              False              False
3              False              False              False
4              False              True              False
```

	Joining Designation_5
0	False
1	False
2	False
3	False
4	False

[5 rows x 45 columns]

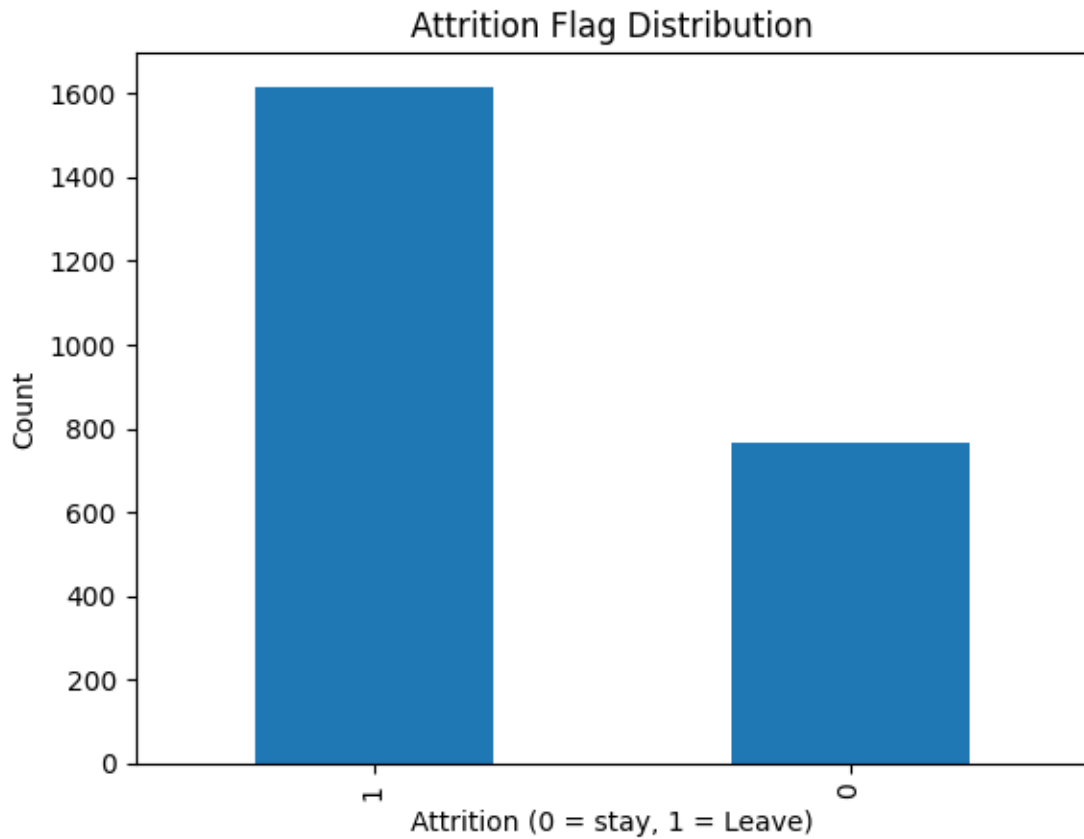
Warning: Total number of columns (45) exceeds max_columns (20) limiting to first (20) columns.

```
[ ]: df_final['Attrition_flag'].value_counts(normalize=True)
```

```
[ ]: Attrition_flag
1    0.678706
0    0.321294
Name: proportion, dtype: float64
```

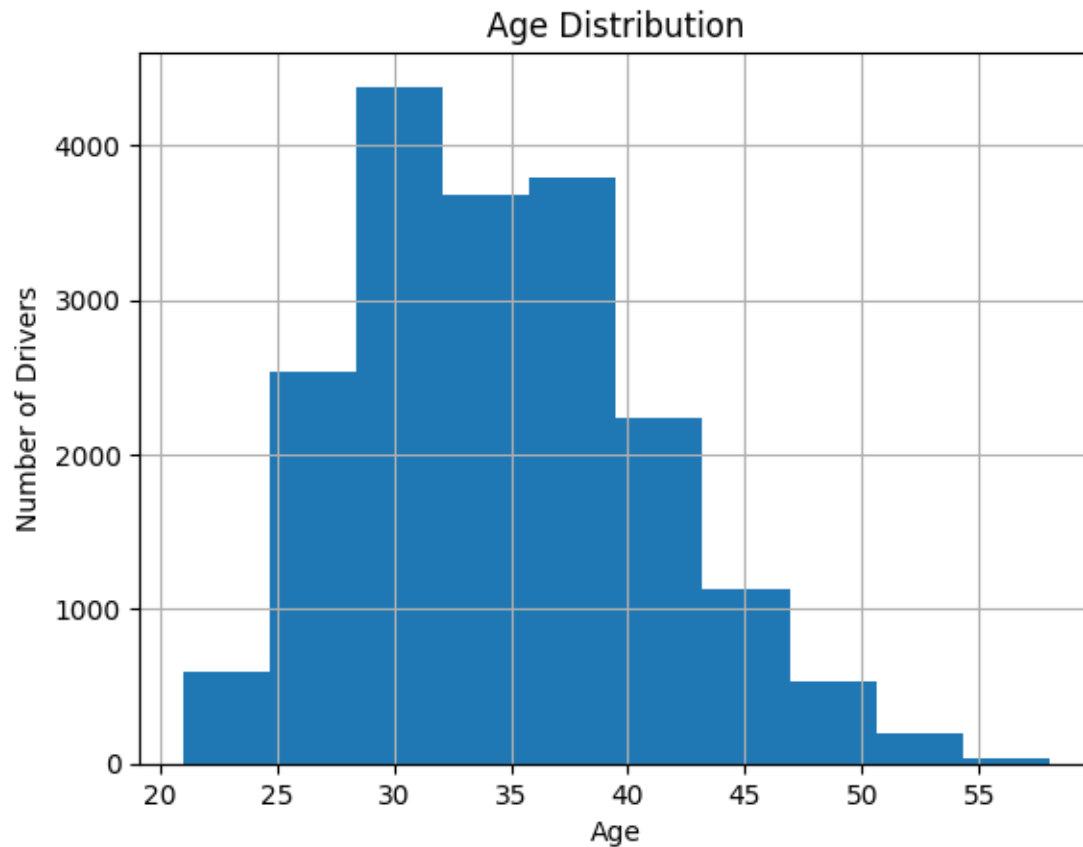
```
[ ]: import matplotlib.pyplot as plt
import seaborn as sns
```

```
[ ]: df_final['Attrition_flag'].value_counts().plot(kind='bar')
plt.title("Attrition Flag Distribution")
plt.xlabel("Attrition (0 = stay, 1 = Leave)")
plt.ylabel("Count")
plt.show()
```



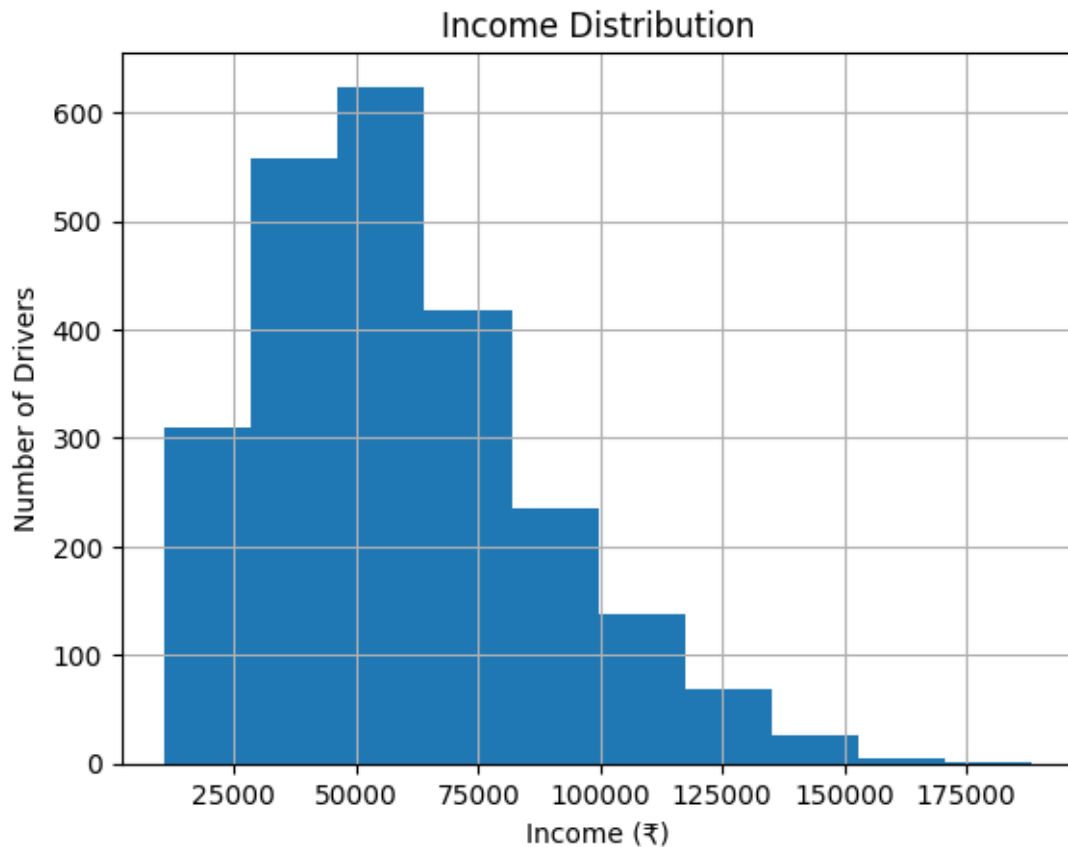
The dataset shows that 67.9% drivers left the company, indicating whether attrition is high or low.

```
[ ]: df['Age'].hist()  
plt.title("Age Distribution")  
plt.xlabel("Age")  
plt.ylabel("Number of Drivers")  
plt.show()
```



most drivers fall in the age range of 30 - 35

```
[ ]: df.groupby('Driver_ID')['Income'].last().hist()  
plt.title("Income Distribution")  
plt.xlabel("Income ( )")  
plt.ylabel("Number of Drivers")  
plt.ticklabel_format(style='plain', axis='x')  
plt.show()
```

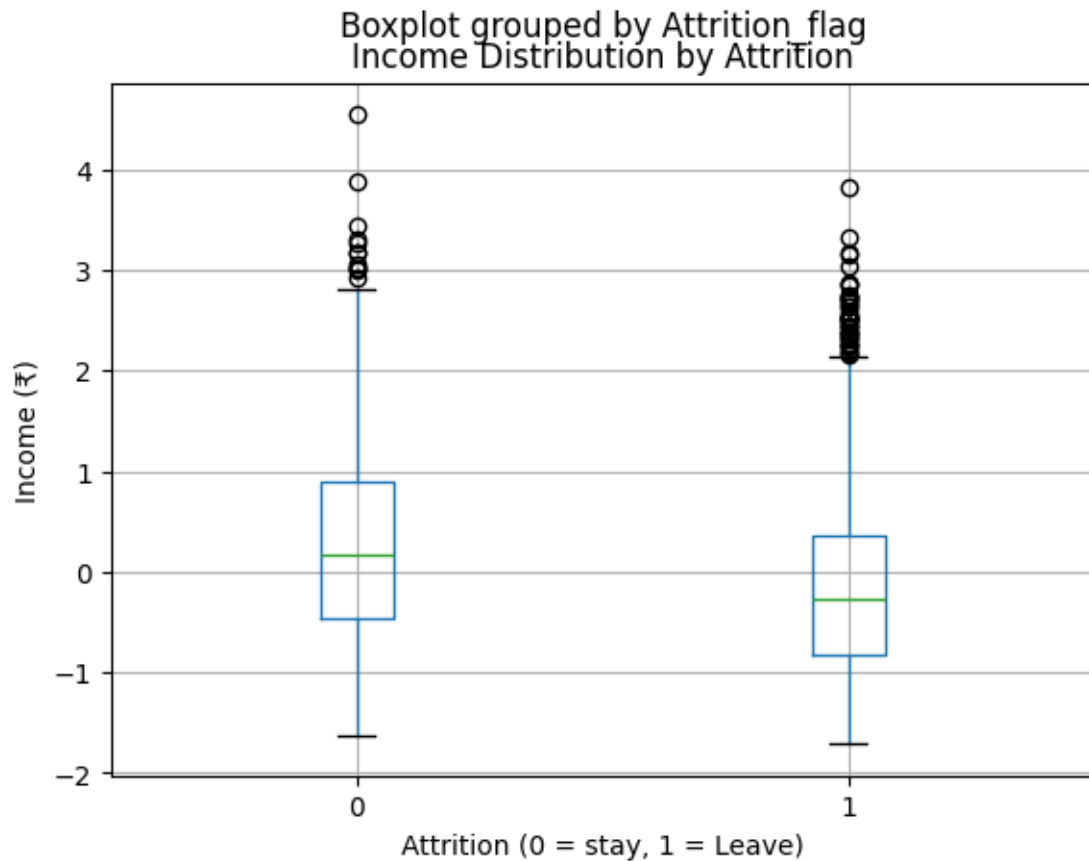


The income distribution shows that most drivers earn within a specific range, with fewer drivers earning significantly higher incomes, indicating a right-skewed distribution.

```
[ ]: df_final.groupby('Attrition_flag')['Income'].mean()
```

```
[ ]: Attrition_flag
0    0.293496
1   -0.138939
Name: Income, dtype: float64
```

```
[ ]: df_final.boxplot(column='Income', by='Attrition_flag')
plt.title("Income Distribution by Attrition")
plt.xlabel("Attrition (0 = stay, 1 = Leave)")
plt.ylabel("Income ( )")
plt.show()
```



Drivers with lower income are more likely to leave (if observed)

```
[ ]: df_final.groupby('Attrition_flag')['Quarterly Rating'].mean()
```

```
[ ]: Attrition_flag
0    1.957077
1    1.381316
Name: Quarterly Rating, dtype: float64
```

```
[ ]: df_final_reconstructed = df.groupby('Driver_ID').agg({
    'Age': 'last',
    'Income': 'last',
    'tenure_days': 'max',
    'Rating_increased': 'sum',
    'Gender': 'last',
    'City': 'last',
    'Education_Level': 'last',
    'Joining Designation': 'last',
    'Grade': 'last',
    'Total Business Value': 'sum',
```

```

        'Quarterly Rating': 'mean',
        'Attrition_flag': 'max',
        'Income_increased': 'sum'
    }).reset_index()

```

```

[ ]: df_final_reconstructed = pd.get_dummies(
    df_final_reconstructed, columns=['City', 'Education_Level', 'Joining_
    ↳ Designation'], drop_first=True)

scaler = StandardScaler()

num_cols = ['Age', 'Income', 'Total Business Value', 'tenure_days']
df_final_reconstructed[num_cols] = scaler.
    ↳ fit_transform(df_final_reconstructed[num_cols])

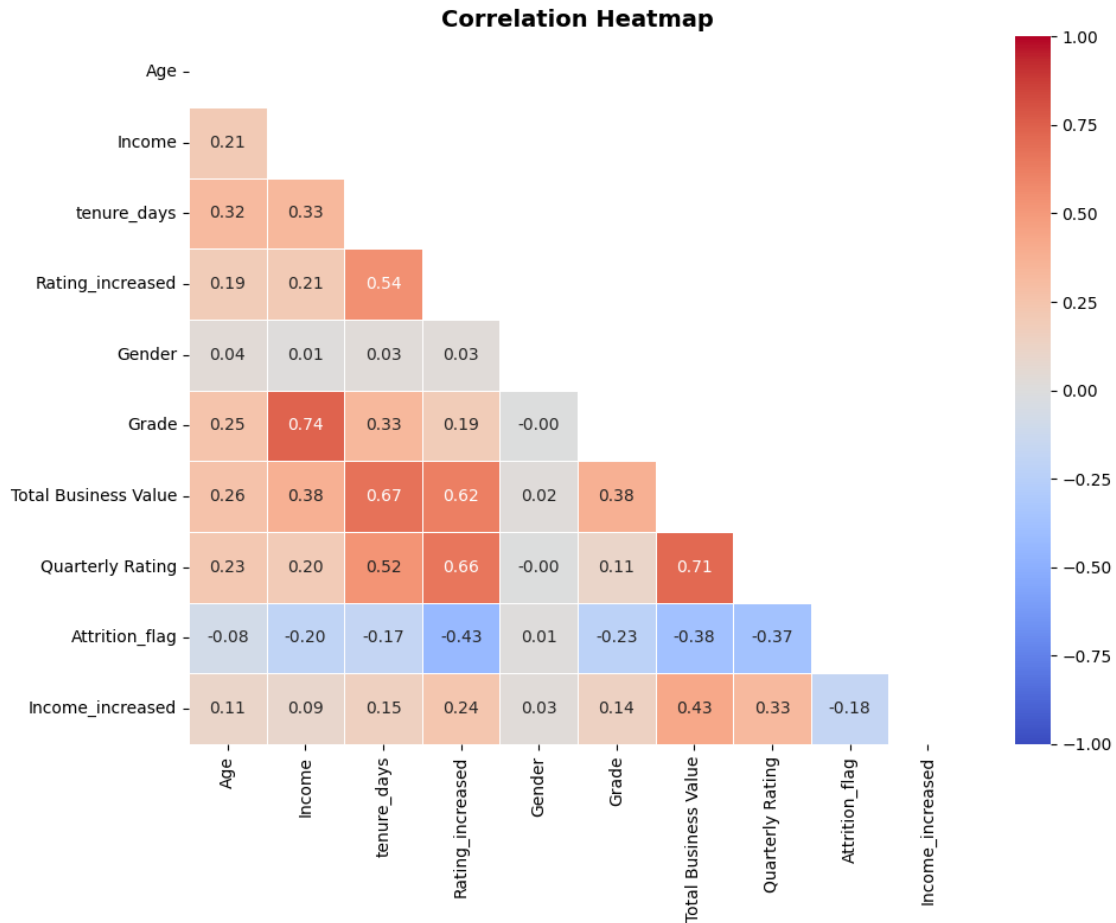
df_final = df_final_reconstructed

numeric_cols_for_corr = df_final.select_dtypes(include=np.number).columns.
    ↳ tolist()
if 'Driver_ID' in numeric_cols_for_corr:
    numeric_cols_for_corr.remove('Driver_ID')

corr = df_final[numeric_cols_for_corr].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, mask=mask, linewidths=0.5, vmin=-1, vmax=1)
plt.title("Correlation Heatmap", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('plot_corr.png', dpi=130, bbox_inches='tight')
plt.show()

```

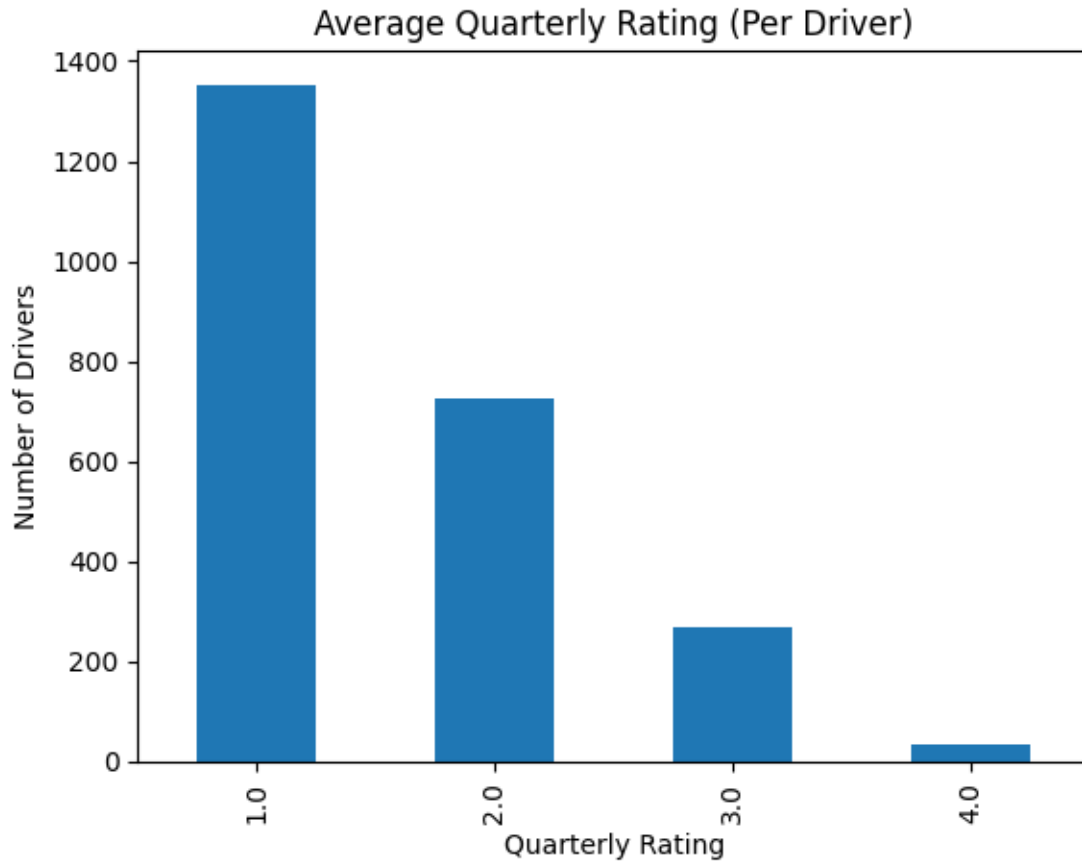


```
[ ]:
```

```
[ ]: df_final.groupby('Attrition_flag')['Rating_increased'].mean()
```

```
[ ]: Attrition_flag
0    1.050980
1    0.272896
Name: Rating_increased, dtype: float64
```

```
[ ]: df.groupby('Driver_ID')['Quarterly Rating'].mean().round().value_counts().
      sort_index().plot(kind='bar')
plt.title("Average Quarterly Rating (Per Driver)")
plt.xlabel("Quarterly Rating")
plt.ylabel("Number of Drivers")
plt.show()
```



The column chart shows the distribution of quarterly ratings, indicating that drivers with lower ratings are more concentrated in the attrition group, while higher-rated drivers tend to stay.

What is the structure of the dataset?

The dataset contains - rows and columns, representing monthly records of drivers

What are the data types of each column?

The data consists of both numeric and categorical attributes. The numeric attributes include Age, Income, Total Business Value, and Quarterly Rating. On the other hand, categorical attributes consist of City, Education Level, and Joining Designation. Date attributes include MMM-YY, Date Of Joining, and LastWorkingDate.

Are there any missing values in the dataset? If so, which columns are affected?

Missing values were observed in several columns. Numerical columns were handled using median imputation, while categorical columns were filled using mode. The LastWorkingDate column was not imputed as it indicates whether a driver is active or has left.

What are the basic statistics (mean, median, standard deviation) for numerical features like Age, Income, Total Business Value, and Quarterly Rating?

The average age of drivers is 30 to 35 years. The average income 50k to 75k. The total business

value shows high variability, indicating differences in driver performance. Quarterly ratings range between 1 and 5, with an average of 1.96

How many unique drivers are there in the dataset?

there are total 2,381 unique drivers in the dataset.

How many drivers joined and left each month?

There are total 1616 drivers left and 17488 drivers join each month.

Can we determine the average tenure of drivers in the dataset

The avg tenure early drivers is < 6 months.

How can we create a target variable to indicate whether a driver has left the company based on LastWorkingDate?

A target variable named Attrition was created using the LastWorkingDate column. Drivers with a non-null LastWorkingDate were marked as 1 (left the company), while those with null values were marked as 0 (active drivers).

What additional features can we extract from Dateofjoining, such as tenure or duration of employment?

Tenure is an attribute developed to capture the time period during which the driver has worked. The reporting date was considered to be the end-of-working-date for active drivers.

What are the distributions of Age, Income, and Total Business Value?

The age distribution shows that most drivers fall within a mid-age range. Income distribution is right-skewed, with a majority of drivers earning lower incomes and a few high earners. Total business value also exhibits high variability, indicating differences in driver performance.

How does Quarterly Rating vary across different drivers and time periods?

Quarterly ratings are generally concentrated in the lower to mid range (1–3). Drivers with consistently low ratings are more likely to leave, while higher-rated drivers tend to remain longer.

Are there any trends or patterns in the monthly income or business value acquired?

Income and business value show variation across drivers and time. Drivers with increasing income and higher business value tend to have better retention, indicating a strong relationship between performance and attrition.

How should missing values in LastWorkingDate be treated, considering it indicates whether a driver has left?

Median imputation was performed for numerical variables, whereas the categorical variables were imputed using the mode. However, for the LastWorkingDate, no imputation was done since it contains active drivers only. The LastWorkingDate was utilized for creating the dependent variable, Attrition.

Is there a correlation between Age and Income?

After analyzing the data, there is no strong relationship between age and income. While one might expect older drivers to earn more due to experience, the dataset does not show a clear trend. This suggests that income is influenced more by performance and activity rather than age.

7. Correlation & Relationship Insights

```
[ ]: # @title

data = {
    "Relationship": [
        "Age vs Income",
        "Education vs Business Value",
        "Rating vs Tenure",
        "Income vs Attrition",
        "Income Growth vs Attrition",
        "Rating Growth vs Attrition"
    ],
    "Finding": [
        "Moderate positive relationship",
        "No strong relationship",
        "Higher ratings → longer tenure",
        "Low income → high attrition",
        "Income growth reduces attrition",
        "Rating improvement reduces attrition"
    ],
    "Impact": [
        "Experience slightly increases earnings",
        "Education not a key factor",
        "Performance drives retention",
        "Income is a key retention factor",
        "Financial growth improves loyalty",
        "Recognition improves engagement"
    ]
}

df_chart = pd.DataFrame(data)
df_chart
```

```
[ ]:
      Relationship                                Finding \
0      Age vs Income      Moderate positive relationship
1 Education vs Business Value      No strong relationship
2      Rating vs Tenure      Higher ratings → longer tenure
3      Income vs Attrition      Low income → high attrition
4 Income Growth vs Attrition      Income growth reduces attrition
5 Rating Growth vs Attrition Rating improvement reduces attrition

                                Impact
0 Experience slightly increases earnings
1      Education not a key factor
2      Performance drives retention
```

- 3 Income is a key retention factor
- 4 Financial growth improves loyalty
- 5 Recognition improves engagement

1. Strategies for Improving Driver Retention

From the above analysis, the main determinant that influences attrition rate is income level. Drivers who make lower amounts of income leave work at a high rate. Thus, implementing measures that ensure minimum pay or improved reward system can aid in ensuring higher retention rates.

Other significant determinants are performance ratings whereby low-performing drivers leave at higher rates. Measures such as training and constructive feedback can improve performance and retention levels.

Finally, tenure is another key determinant since most drivers leave prematurely. Bonuses when attaining milestone periods like 6 months or 1 year can ensure improved retention rates.

2. Targeted Interventions

Those drivers who are not demonstrating any income growth and have low ratings must be targeted. This is because such drivers tend to leave the company. The organization can use this knowledge and offer certain incentives to these drivers.

Besides, considering that the attrition rate differs between different cities, the organization can develop location-based solutions.

Overall, the analysis shows that driver attrition is not random. It is strongly influenced by factors like income, performance, and growth opportunities. By focusing on these areas, Ola can reduce attrition, improve driver satisfaction, and lower the cost of acquiring new drivers.

```
[ ]: !jupyter nbconvert --to pdf ola.ipynb
```

```
[ ]:
```

```
[ ]:
```